

Aula – Isolando Dependências com Mocks e Spies

Objetivo: Entender e aplicar isolamento de dependências em testes

1. Objetivo da Aula

“Hoje vamos aprender algo que muda completamente o nível de testes:

 testar código sem depender do mundo externo”

2. Problema (comece pela dor)

 **Fala importante**

“Imagina que seu código depende de:

- banco de dados
- API externa
- serviço lento

Você quer isso rodando no seu teste?”

(pausa)

 “Claro que não.”

Conceito chave

Dependência = algo que seu código precisa para funcionar

Exemplo:

service → repository → banco de dados

🌟 Problema na prática (exemplo simples)

Código

```
class EmailService {
  enviar(email: string) {
    console.log('Enviando email para:', email);
  }
}

class UserService {
  constructor(private emailService: EmailService) {}

  criarUsuario(email: string) {
    this.emailService.enviar(email);
  }
}
```

Pergunta

👉 “Se eu testar isso... o que acontece?”

Resposta:

- Ele realmente envia email
- Ou tenta simular envio
- Pode ser lento ou imprevisível

Insight

👉 “Seu teste ficou dependente de algo externo”

🔥 Solução: Isolar dependências

“Em teste, você controla o ambiente”

5. O que são Mocks e Spies

Spy (espiar)

 Observa o comportamento real

Exemplo:

- verificar se função foi chamada
- verificar quais dados foram enviados

Mock (simular/substituir)

 Substitui o comportamento real

Exemplo:

- não chamar banco
- não enviar email
- devolver valores controlados

6. Explicação visual

Sem mock:

UserService → EmailService → mundo real

Com mock:

UserService → Mock → resultado controlado

7. Exemplo com SPY

Código

```
import { jest } from '@jest/globals';

class EmailService {
  enviar(email: string) {
    console.log('Enviando email para:', email);
  }
}
```

Teste com spy

```
it('deve chamar o método enviar email', () => {
  const emailService = new EmailService();

  const spy = jest.spyOn(emailService, 'enviar');

  emailService.enviar('teste@email.com');

  expect(spy).toHaveBeenCalled();
});
```

Explicação

“Eu não mudei o comportamento.

Eu só observei se ele foi chamado.”



8. Exemplo com MOCK

Criando mock manual

```
const mockEmailService = {  
  enviar: jest.fn()  
};
```

Testando com mock

```
it('deve chamar o envio de email ao criar usuário', () => {  
  const mockEmailService = {  
    enviar: jest.fn()  
  };  
  
  const userService = new UserService(mockEmailService as any);  
  
  userService.criarUsuario('teste@email.com');  
  
  expect(mockEmailService.enviar).toHaveBeenCalled('teste@email.com');  
});
```

Explicação

“Agora eu substituí o serviço real.

Nenhum email foi enviado.”

9. Diferença clara (muito importante)

Conceito	O que faz
Spy	observa comportamento real
Mock	substitui comportamento

10. Simulando cenários com mock

Exemplo avançado

```
const mockRepo = {  
  salvar: jest.fn().mockImplementation(() => {  
    return { id: 1, nome: 'Teste' };  
  })  
};
```

Explicação

 “Agora eu controlo o retorno”

13. Momento ouro da aula

Fala forte:

“Agora você consegue testar qualquer cenário.

Mesmo sem banco

Mesmo sem API

Mesmo sem sistema real”

14. Erros comuns (alerta importante)

- não usar mock quando deveria
- depender de implementação real
- não verificar chamadas
- misturar responsabilidade

15. Fechamento

“Mocks e spies te dão controle total sobre o teste.

E controle é o que separa testes frágeis de testes profissionais.”

16. Resumo Final

👉 Mock = substitui

👉 Spy = observa

👉 Objetivo = isolar dependências